

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction:

The recruitment process today is often time-consuming, inconsistent, and heavily dependent on manual resume screening. Employers must review large volumes of applications, making it difficult to fairly evaluate candidates based on their actual skills and experience. Additionally, traditional systems rely heavily on self-reported information, which may not always be accurate or verifiable.

*To address these challenges, the **Context-Based Application Tracking System (CATS)** is proposed as an AI-powered solution that automates and enhances candidate screening. The system integrates a multi-agent pipeline driven by an LLM model to transform unstructured resume data into structured, verified, and evaluated candidate profiles.*

1.1. Purpose:

This document defines the system architecture, technical design, and operational workflow of the CATS system. It provides a clear reference for developers, testers, and stakeholders to understand system functionality, data processing, and AI integration.

So, the key element of system that has been covered are as followed:

1. Architecture:

- a. Layered architecture consisting of Presentation, Application, AI Processing, and Data layers.

2. Data Flows:

- a. End-to-end movement of data from user input to AI processing and output display.
 - Employer → submits job → stored in database
 - Applicant → uploads resume → stored in database
 - Orchestrator → retrieves job + applicant data
 - AI agents → process and enrich data step-by-step
 - Final results → stored back in database
 - Employer UI → retrieves and displays ranked candidates

3. Model Process:

- a. Multi-agent pipeline handling parsing, enrichment, verification, and evaluation.

4. Role of Reference:

- a. Serves as a technical reference for system development and validation.

1.2. Background:

Traditional applicant tracking systems rely heavily on keyword matching and manual screening, limiting their ability to accurately assess candidates. They also lack verification mechanisms and intelligent insights.

The CATS system improves this by introducing AI-driven parsing, cross-source verification, automated ranking, and dynamic interview question generation. This enhances accuracy, efficiency, and fairness in recruitment.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
<i>Employer / HR</i>	<i>Post jobs, review candidates, make hiring decisions.</i>	<i>Faster recruitment cycle, reduced workload in manually screening CVs, high-quality candidate recommendations, and less human bias during the selection process.</i>
<i>Candidates</i>	<i>Submit applications and receive evaluation.</i>	<i>Fair and unbiased evaluation, quick response from the system, easy and user-friendly application process, and clear, constructive feedback on their performance.</i>
<i>Development Team</i>	<i>Develop and maintain system</i>	<i>Immediate notification of the food ordered correct order information.</i>
<i>Development Team</i>	<i>Build the platform and maintain system.</i>	<i>Clear architecture and modular design.</i>
<i>QA Team</i>	<i>Validating the system behavior for all the user roles.</i>	<i>Reliable outputs and full test Coverage.</i>

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web-Based Application
Architecture	Layered Architecture with AI Integration

The CATS system is structured as a layered architecture consisting of four main components: Presentation Layer, Application Layer, AI Processing Layer, and Data Layer.

The Presentation Layer includes the Employer Dashboard and Candidate Portal, which allow users to interact with the system by submitting job postings, uploading resumes, and viewing evaluation results. These interfaces communicate with the Application Layer through HTTP requests.

The Application Layer acts as the central controller of the system. It handles business logic, manages workflows, and coordinates communication between the frontend, AI processing components, and the database. It ensures that user requests are properly processed and routed to the appropriate services.

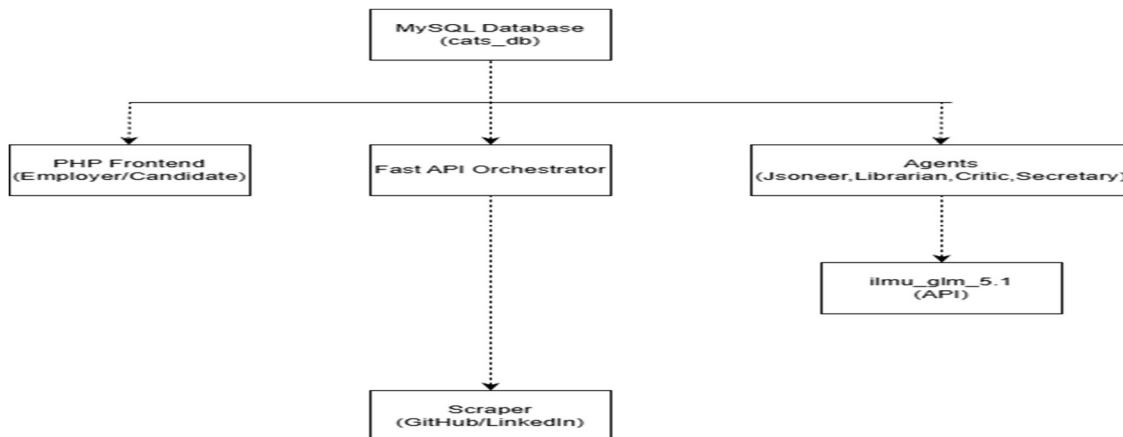
The AI Processing Layer is the core intelligence component of the system. It executes a multi-agent pipeline powered by the LLM, where each agent performs a specific task such as resume parsing, data enrichment, skill verification, evaluation, and interview question generation.

The Data Layer is responsible for storing and managing all system data. It uses MySQL with JSON support to store structured data such as job information and unstructured data such as resumes and AI-generated outputs. This layer ensures data integrity, persistence, and efficient retrieval.

2.1.2 LLM as Service Layer

In CATS, the LLM layer is the core engine of the system. Every agent communicates with **ilmu-glm-5.1** to perform its function. The parser, librarian, critic, and secretary all depend on the LLM for their operations.

2.1.3 Dependency Diagram

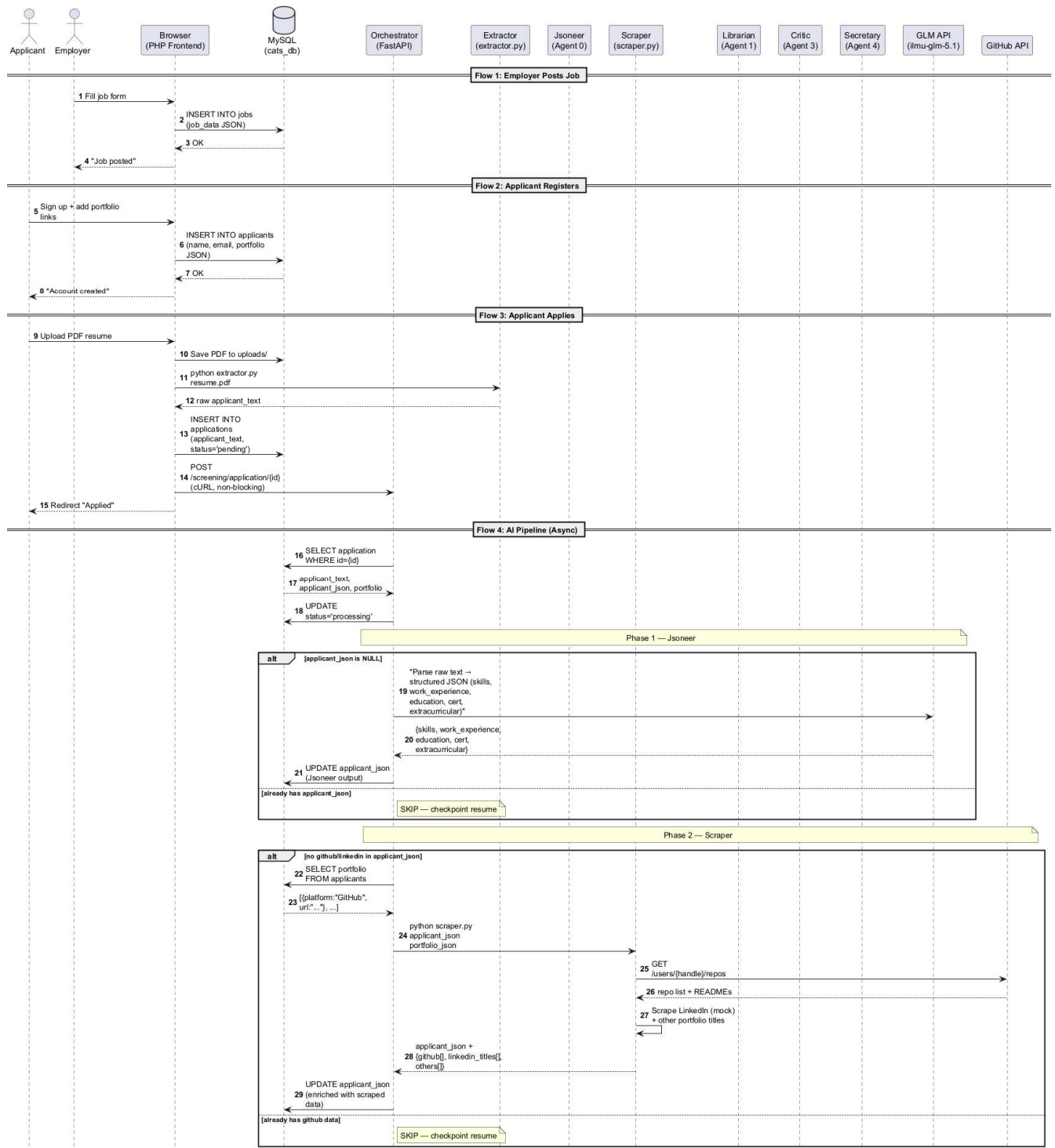


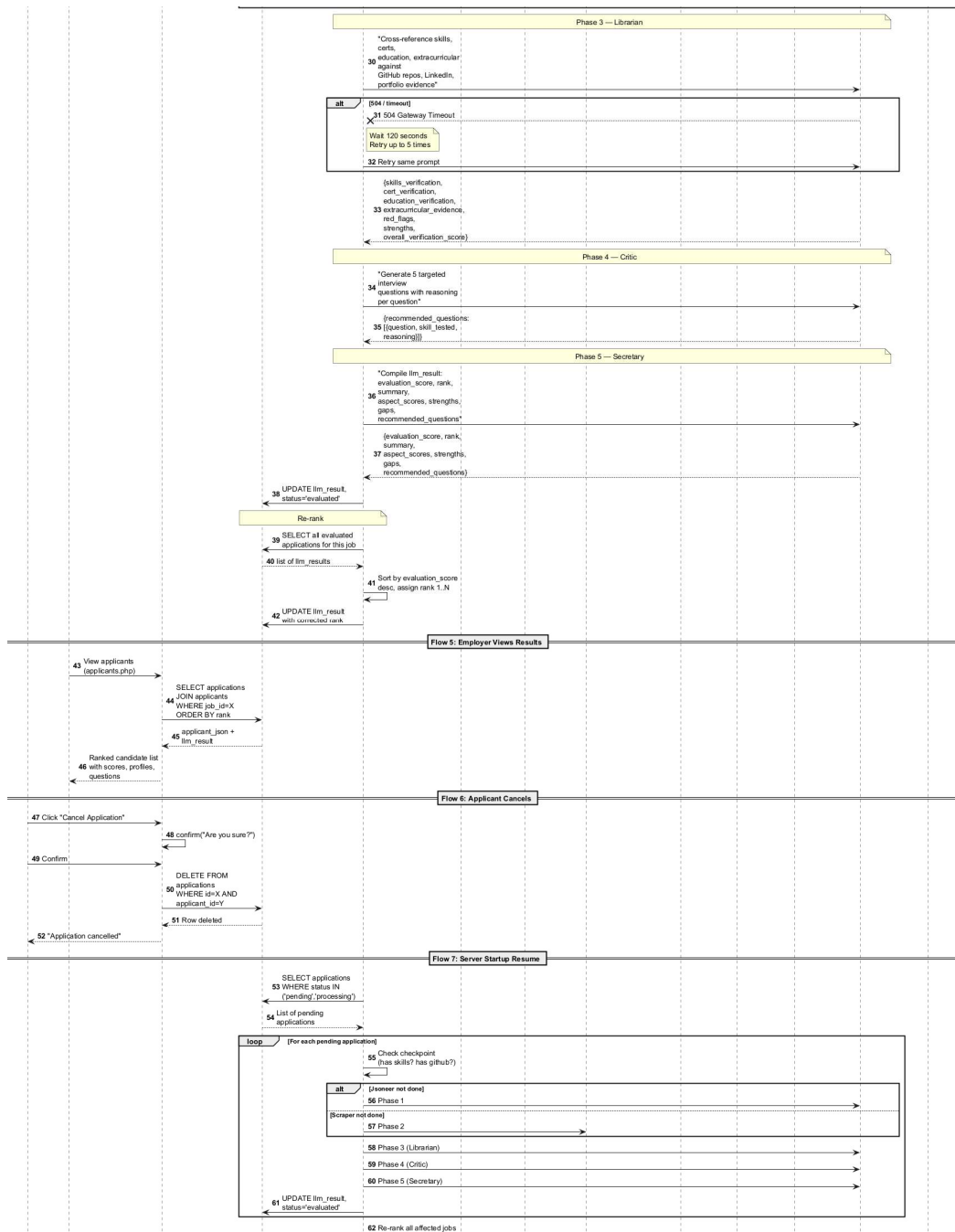
- a. The Orchestrator constructs prompts by combining a system role, task instructions, input data, and a strict JSON output format to ensure consistent responses. Each prompt is tailored to a specific pipeline stage, such as parsing, verification, or evaluation. The constructed prompt is then sent to the GLM API through an HTTP POST request, including parameters like model name, temperature, and maximum tokens. This ensures controlled and predictable AI behavior across different processing phases.
- b. The context window contains all relevant data required for a specific task, including resume text, structured applicant data, and external portfolio information. In earlier stages, it mainly includes raw resume content, while later stages incorporate enriched data such as GitHub repositories and verification results. The final evaluation stage uses the largest context window, combining all previous outputs to generate scores, rankings, and summaries. This progressive accumulation of data allows the system to make more informed decisions.
- c. The system receives responses from the GLM API in JSON format through the Orchestrator. It extracts the response content, converts it into structured data, and validates it against the expected schema. After validation, the data is stored in the database and passed to the next stage of the pipeline. Each component in the pipeline uses the previous output as its input, ensuring a continuous and structured flow of information.
- d. Token limits are enforced within the Orchestrator before sending any request to the GLM API. The system checks the size of the input data and compares it against the model's context limit. If the input exceeds the limit, it is split into smaller chunks, such as dividing the resume into sections. Each chunk is processed separately, and the results are later merged to form a complete output.
- e. The frontend handles user interactions and sends requests to the backend system. The backend, managed by the Orchestrator, processes these requests and

coordinates the workflow. It communicates with the database to ensure persistent storage of application data and results. At the same time, it interacts with external services such as the GLM API for AI processing and the Scraper service for retrieving additional applicant data.

- f. The system is designed as a distributed architecture where each component operates independently. The frontend, API gateway, Orchestrator, Scraper service, database, and GLM API are deployed as separate services. These components communicate through HTTP-based APIs, allowing independent scaling and maintenance. This distributed design improves system reliability, scalability, and fault tolerance.

2.2.2. Sequence Diagram





1. Employer posts job
2. Applicant registers
3. Applicant applies (triggers pipeline)
4. AI pipeline — Orchestrator → Jsoneer → Scraper → Librarian → Critic → Secretary → Orchestrator
5. Employer views ranked results

2.2. Technological Stack

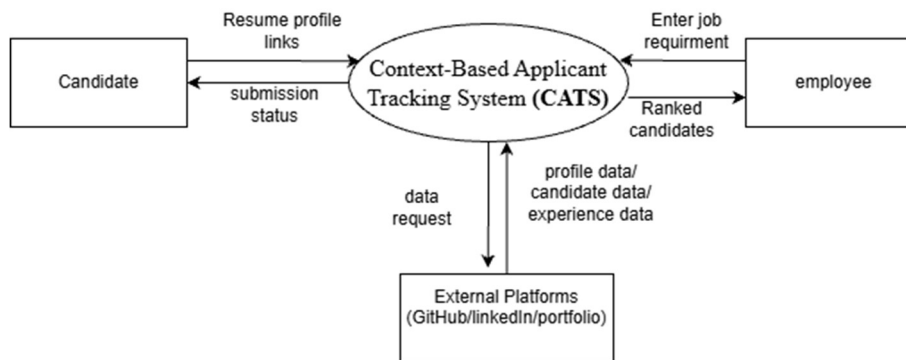
- 2.2.1. Frontend [PHP, HTML, CSS]
- 2.2.2. Backend [Python, APACHE]
- 2.2.3. Database [MySQL + JSON]
- 2.2.4. Ai Model [ilmu-glm-5.1]
- 2.2.5. Hosting [Local / Cloud-ready]

2.3. Key Data Flows

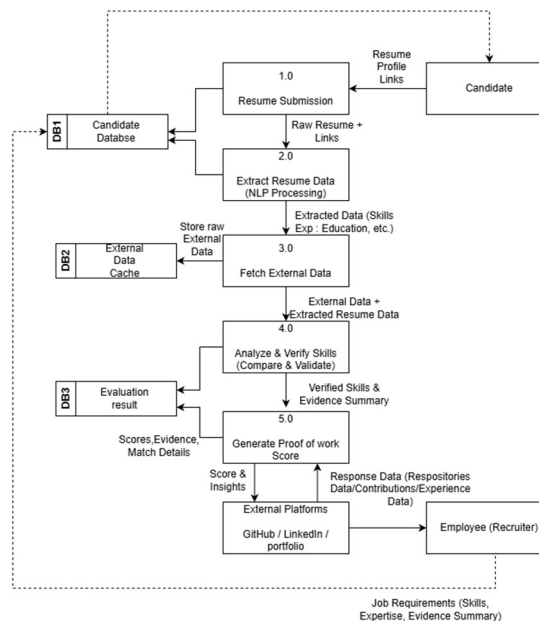
2.3.1. Here, do show how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)

a. Level 0 (DFD)

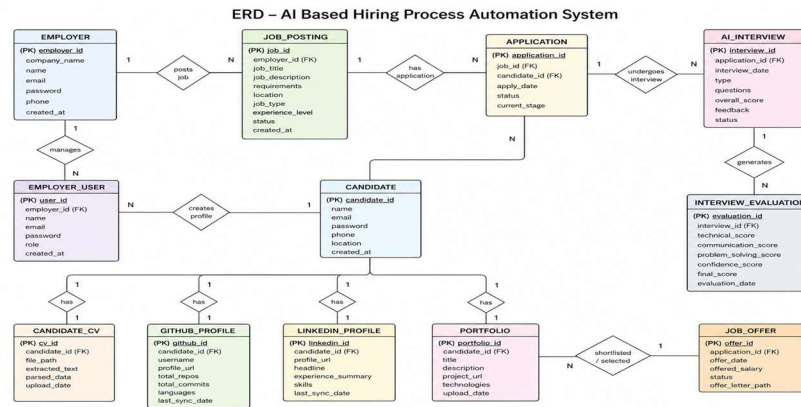


b. Level 1 (DFD)



2.3.2. Normalized Database Schema

a. ERD diagram



3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. The following core features will be built and demonstrated by the end of UMHackathon 2026:

#	Features	Description
1	Skill Verification	Cross-checks claimed skills with real evidence from GitHub repositories and online profiles. The Librarian agent verifies each claimed skill against scraped data and assigns a verification score with reasoning.
2	Automated Interview Question Generation	Produces five personalized, scenario-based interview questions for each candidate based on their specific evidence gaps and job requirements, not generic questions.
3	Employer Dashboard	Displays ranked candidates with comprehensive insights including overall score, aspect breakdowns (technical skills, experience, certifications, community engagement, project quality), strengths, gaps, and recommended questions.
4	Data Enrichment	Integrates external data from GitHub (repositories, languages, commit history) and LinkedIn (profile

		<i>information) to enhance applicant profiles beyond what is stated in the resume.</i>
5	<i>End-to-End AI Pipeline</i>	<i>Complete multi-agent processing from resume upload to final evaluation, demonstrating the Jsoneer → Scraper → Librarian → Critic → Secretary workflow.</i>

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system.

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g.)
Scalability	Support 500 applications without slow queries.	Add indexes on applications.job_id and applications.status . Use LIMIT with pagination on employer applicants page (50 per page). Consider partitioning applications by created_at month if needed.
Reliability	95% of applications process without manual intervention	Each agent validates LLM output before passing to next. If JSON parsing fails, agent retries once with a corrected prompt. Malformed output logged to file. Application marked "failed" with error reason.
Maintainability	Each agent can be modified independently	[Each agent in separate file (jsoneer.py , librarian.py , etc.). Each has a clearly defined input schema and output schema. No shared global state between agents. Prompts hardcoded inside each agent file for easy tweaking.
Usability	Candidate applies in under 3 minutes	Registration form asks only essential fields (name, email, password, portfolio links). Resume upload supports drag-and-drop. Application confirmation shown immediately.

		<i>Status page shows "processing" or "complete".</i>
<i>Token Latency</i>	<i>API calls to GLM must return response within reasonable time</i>	<i>Implement timeout of 30s for API calls. Display user-facing loading state while awaiting response. Process applications asynchronously to prevent UI blocking.</i>

3.3. Out of Scope / Future Enhancements

- **Real-time LinkedIn Integration**
Full API-based integration for accurate professional data retrieval.
- **Advanced NLP Resume Parsing**
Use of deep learning models for more accurate and context-aware data extraction.
- **Video Interview Integration**
Automated interview scheduling and AI-based video analysis.
- **Multi-language Support**
Ability to process resumes in different languages.
- **Mobile Application**
Dedicated mobile app for applicants and employers.
- **Advanced Analytics Dashboard**
Insights such as hiring trends, skill demand, and candidate comparison.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing: *The system can be deployed using a grayscale rollout approach, where it is gradually introduced to a small group of employers before being released to all users. For example:*

Phase 1: Deploy to 5% of employers with monitoring

Phase 2: Increase to 25% if no critical issues detected

Phase 3: Increase to 50%, then 100%

A/B testing can be conducted by comparing two different versions of the AI evaluation model.

By analyzing metrics such as:

Ranking accuracy (employer satisfaction with top candidates)

Processing time per application

Question relevance score (employer feedback)

The team can determine which model version performs better before full deployment.

4.1.2. Emergency Rollback (ER) & Golden Release: A recovery process used when a newly deployed version causes serious problems. If the AI evaluation module produces incorrect candidate scores or the system becomes unstable after an update, an emergency rollback will restore the last working version to minimize downtime and prevent data corruption.

Golden Release: A fully tested and stable version of the system considered safe for deployment. It acts as a backup version that the system can always return to if issues occur in newer releases. The golden release includes:

Functional CV screening

Working AI pipeline with all five agents

Stable database integration

Verified evaluation output format

4.1.3. Priority Matrix:

A Priority Matrix is used to categorize tasks, bugs, or system issues based on their level of urgency and importance. It helps the development team decide which problems should be solved first. Issues that are both urgent and highly important, such as errors in AI scoring that affect hiring decisions, must be fixed immediately. Problems that are urgent but less important, such as minor interface bugs, can be resolved quickly but are not critical. Tasks that are important but not urgent, such as adding new features or improvements, are planned for future updates. Low-importance and low-urgency tasks, such as small design changes, are given the lowest priority.

4.2. Monitoring:

Monitoring refers to the continuous observation of system performance, security, and overall functionality to ensure that the system operates smoothly. In this hiring system, monitoring includes checking server performance, system uptime, database activity, AI interview response accuracy, and user interactions. It also involves tracking error logs and detecting any unusual system behavior. Monitoring is important because it helps identify problems early, improves system reliability, and ensures that the hiring process runs efficiently without interruption. For example, if the AI interview system becomes slow or the database fails to respond, the monitoring system will trigger an alert so that immediate action can be taken.

4.3. Assumptions:

- a. **Infrastructure:** All users have stable internet access and devices capable of running a web-based application.
- b. **Data Accuracy:** Candidates provide accurate and truthful information in their CVs, GitHub profiles, LinkedIn accounts, and portfolios.
- c. **External API Availability:** GitHub API and AI/NLP services (ilmu-glm-5.1) are available, stable, and accessible during operation.

- d. **Data Consent:** Candidates consent to their data being processed and analyzed by the system for recruitment purposes.
- e. **AI Fairness:** The AI model is sufficiently trained to evaluate candidates fairly based on available data without systematic bias.
- f. **Employer Input Quality:** Employers correctly define job requirements and evaluation criteria to ensure accurate matching of candidates.

4.4. External Dependencies:

Tools	Purpose	Risks
Z.AI GLM API (ilmu-glm-5.1)	Core LLM reasoning engine for prompt inference, multi-step reasoning, and structured JSON output generation.	High - If GLM API experiences downtime or rate limiting, the entire core product fails. No processing can occur without the LLM.
GitHub API	Retrieve candidate's technical profile including repositories, programming languages, commit history, and project contributions.	Medium - If GitHub API is down or rate-limited, or if candidate's profile is private, evaluation may be incomplete.
LinkedIn	Extract professional information such as education, work experience, certifications, and skills.	Medium - LinkedIn has strict API access policies. Full data access may not be allowed. Data may be outdated. Privacy concerns.
MySQL Database	Store all structured and JSON data for users, jobs, applications, and audit logs.	Low - Database is locally hosted and controlled. Standard backup procedures apply.

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

- Planning & Requirement Analysis - (Day 1 & 2)
- System Design - (Day 3)
- Database Setup & Frontend Development (Basic) - (Day 4 & 5)

- Receive LLM API key – (Day 5)
- Resume Upload & Extraction & Data Enrichment - (Day 6)
- AI Pipeline Development - (Day 7)
- Evaluation & Ranking - (Day 8)
- Integration & Testing - (Day 9)
- Documentation & Presentation - (Day 10)

5.2. Team Members Role:

Team Member	Role	Responsibilities
Prem Ishwar	<i>Front End Developer</i>	<i>Develops and configures AI components, designs agent prompts, tests LLM output quality, and optimizes evaluation logic.</i>
Syamendrera Ruben	<i>Backend Developer (Python/Ilmu GLM)</i>	<i>Develops the AI orchestrator, implements the processing pipeline, manages FastAPI endpoints, and handles LLM API integration.</i>
Puteri Nur Aisyah	<i>Data Analyst</i>	<i>Analyze and configure JSON into an AI agent friendly format.</i>
Siti Aishah	<i>Documentation Specialist</i>	<i>Prepares and maintains all project documentation, ensures consistency across documents, and manages version control.</i>
Nurul Nazirah	<i>Technical Documentation Writer</i>	<i>Records system architecture, creates data flow diagrams, documents methodologies, and describes system features in the SAD.</i>

5.3. Recommendations:

- **Performance & Scalability:**

Implement Redis caching for frequently accessed job data and candidate lists to reduce database load.

Add asynchronous processing with task queues (e.g., Celery) to handle multiple applications concurrently.

Implement connection pooling for both MySQL and LLM API calls.

- **Reliability:**

Add retry logic with exponential backoff for LLM API calls.

Implement fallback mechanisms when individual agents fail.

Create automated health checks for all external dependencies.

- **AI Quality:**

Implement prompt versioning to track and roll back prompt changes.

Add evaluation consistency testing (same resume processed multiple times should yield similar scores).

Develop a feedback loop where employer ratings of candidates can improve model prompts.

- **Security**

Implement file validation to prevent malicious PDF uploads.

Add rate limiting on application submissions.

Encrypt sensitive data in the database (PII fields).

Implement proper session management and CSRF protection.

- **User Experience:**

Add real-time processing status updates (WebSocket or polling).

Implement notification system for when evaluations complete.

Add filtering and sorting options on employer dashboard.

Provide candidates with a summary of their evaluation (if consented).